Project - High Level Design

On

Multi-Agent Education System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Aditya Paliwal	EN22CS301055
2	Abhiket Thakur	EN22CS301030
3	Akash Kushwaha	EN22CS301083
4	Aarushi Mittal	EN24CS3T10026
5	Akshita Rathore	EN22CS301100
6	Akshay Dashore	EN22CS301096

Group Name: 04D1

Project Number: AAI-04

Industry Mentor Name:

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The Multi-Agent Education System is an AI-powered platform designed to generate structured and high-quality educational study guides based on user-provided topics. The system leverages multiple intelligent agents, each with a specific role, to collaboratively process and transform raw input into well-organized learning content.

With the growing demand for personalized and efficient learning tools, this system aims to simplify the process of understanding complex topics by automating research and content generation. By integrating advanced language models and a modular multi-agent architecture, the platform ensures accuracy, clarity, and scalability in content delivery.

1.1 Scope of the document

This document presents the High-Level Design (HLD) of the Multi-Agent Education System, an AI-powered platform that generates structured educational study guides from user-provided topics.

It outlines the system architecture, key components, data flow, and interactions between modules, including the multi-agent workflow implemented using CrewAI and the integration of the Groq LLaMA 3.1 language model.

The document focuses on:

- Overall system structure and design decisions
- Responsibilities of core components (UI, orchestrator, agents, storage)
- End-to-end request processing flow
- Technology stack and system organization

1.2 Intended Audience

This document is intended for stakeholders involved in the design, development, and evaluation of the Multi-Agent Education System.

The primary audience includes:

- Developers and Engineers – to understand the system architecture, components, and overall design for implementation and maintenance.
- Project Reviewers / Evaluators – to assess the system design, approach, and technical decisions.
- AI/ML Practitioners – to gain insights into the multi-agent workflow and integration of large language models.

- Product Designers / Managers – to understand system capabilities and potential enhancements.

1.3 System Overview

The system is structured as a multi-agent pipeline that processes user input and generates educational content through coordinated task execution.

A user interacts with the system via a Streamlit-based interface by providing a topic. This input is handled by the application layer, which invokes a CrewAI-based orchestration module. The orchestrator manages the execution of predefined tasks in a sequential manner.

The workflow begins with the Researcher Agent, which collects and organizes relevant information. The output is then passed to the Writer Agent, which formats the content into a structured study guide. Both agents utilize a language model (Groq LLaMA 3.1) for processing.

The generated output is displayed to the user and stored locally for future access. The system maintains a simple storage mechanism using markdown files and a JSON-based history.

The design supports modularity, enabling easy extension of additional agents or functionalities without major architectural changes.

2 System Design

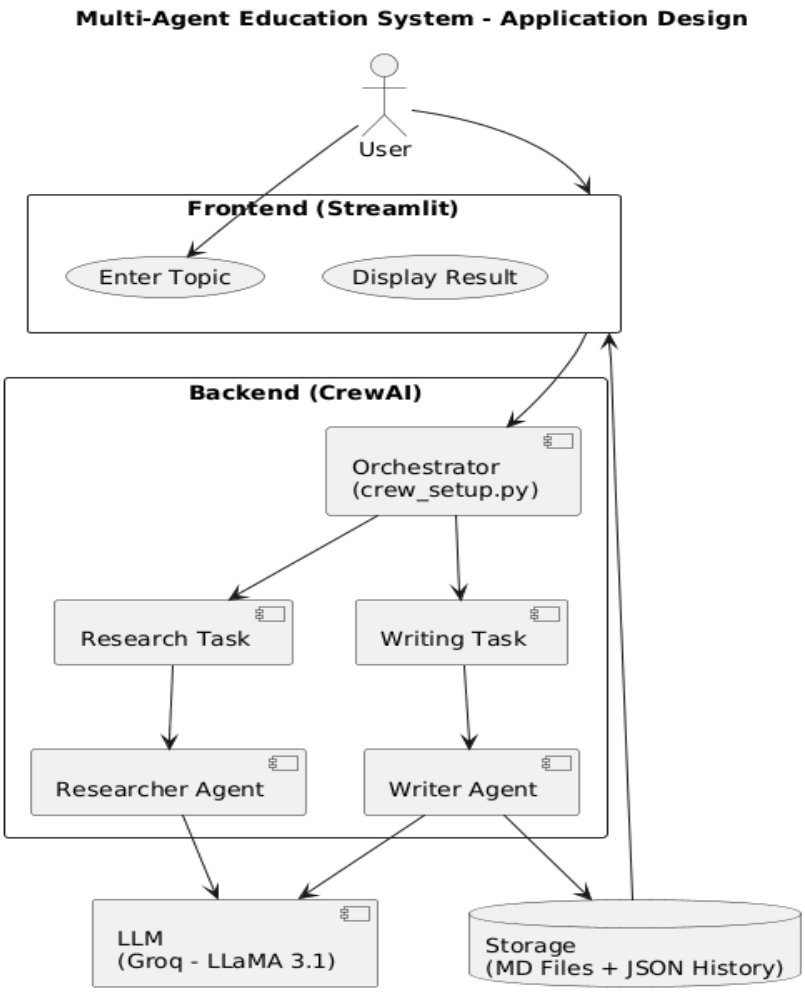
The system follows a multi-agent architecture with a Streamlit-based frontend and a CrewAI backend. User input is processed by an orchestrator that executes agents sequentially.

The Researcher Agent gathers information, and the Writer Agent converts it into a structured study guide using a language model (Groq LLaMA 3.1). The output is displayed to the user and stored locally. The design is modular, allowing easy addition of new agents or features.

2.1 Application Design

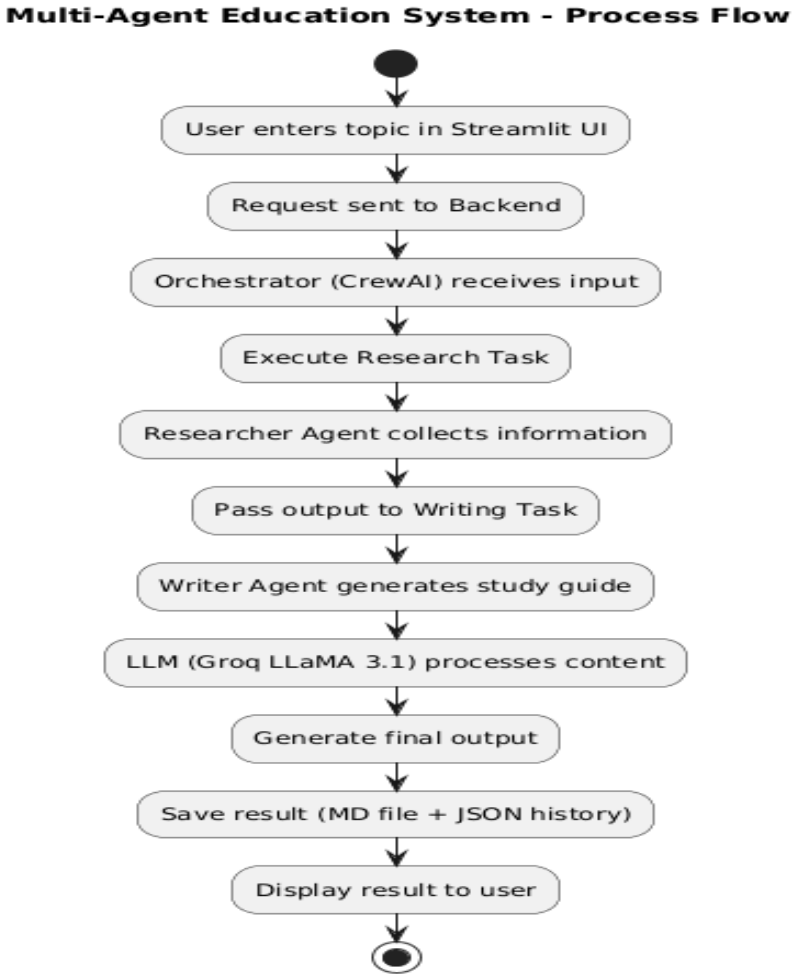
The application is divided into a frontend and backend. The frontend, built using Streamlit, handles user interaction and displays results.

The backend uses CrewAI to manage agent execution. It consists of an orchestrator, task definitions, and agents that process input sequentially. The system integrates a language model for content generation and uses local storage for saving outputs and history.



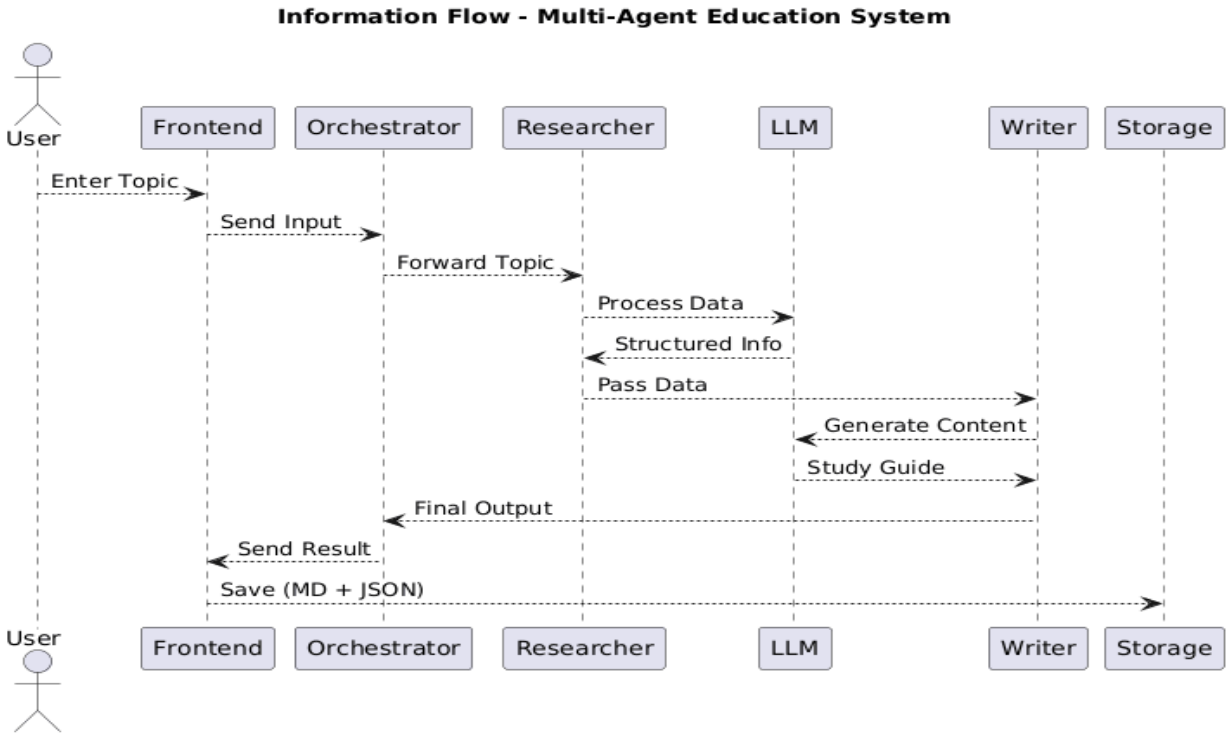
2.2 Process Flow

- User enters a topic through the Streamlit interface
- Input is sent to the backend system
- CrewAI orchestrator receives and processes the request
- Research Task is executed by the Researcher Agent
- Research output is passed to the Writing Task
- Writer Agent generates a structured study guide using the LLM
- Final output is generated
- Result is saved as a markdown file and stored in JSON history
- Generated study guide is displayed to the user



2.3 Information Flow

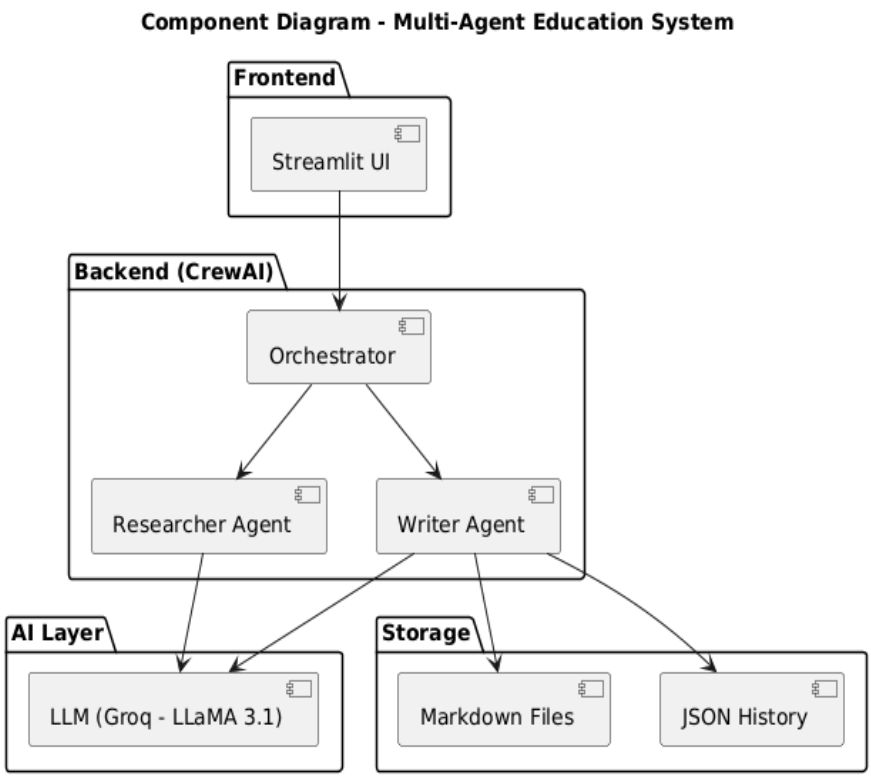
The information flow begins when the user enters a topic through the frontend interface. This input is forwarded to the backend orchestrator, which manages the execution of agents. The orchestrator sends the input to the Researcher Agent, which processes it and generates structured information. This information is then passed to the Writer Agent, which converts it into a well-formatted study guide using the language model. The processed output is returned to the orchestrator and then sent back to the frontend for display. Simultaneously, the generated content is stored in local storage as markdown files and recorded in a JSON-based history for future access.



2.4 Components Design

The system is composed of multiple modular components, each responsible for a specific function:

- Frontend (Streamlit)
  Handles user input and displays the generated study guide.
- Orchestrator (CrewAI)
  Manages the workflow and coordinates execution of agents and tasks.
- Researcher Agent
  Collects and structures information related to the input topic.
- Writer Agent
  Converts structured data into a well-formatted study guide.
- LLM (Groq LLaMA 3.1)
  Provides language understanding and content generation capabilities.
- Storage Layer
  Saves generated outputs as markdown files and maintains history in JSON format.



2.5 Key Design Considerations

- **Modularity** - The system is designed with separate components (agents, tasks, frontend) to ensure easy maintenance and extension.
- **Scalability** - The architecture allows adding new agents or features without major changes to the existing system.
- **Sequential Workflow** - Tasks are executed in a controlled sequence to maintain logical flow and consistency in output.
- **Simplicity** - A lightweight design using local storage and minimal dependencies ensures ease of development and deployment.
- **Reusability** - Agents and tasks are designed to be reusable across different topics and use cases.
- **Extensibility** - The system can be extended to include additional agents such as quiz generators or recommendation systems.

2.6 API Catalogue

The system uses internal function calls and framework-based interactions instead of traditional REST APIs. The following are the key interfaces used within the system:

- **Generate Study Guide API**
  - Description: Initiates the multi-agent workflow for a given topic
  - Input: Topic (string)
  - Output: Generated study guide (string/markdown)
  - Handler: crew.kickoff(inputs={"topic": topic})
- **Load History API**
  - Description: Retrieves previously generated study guides
  - Output: List of history entries (JSON)
  - Handler: load_history()
- **Save to History API**
  - Description: Stores generated content in history
  - Input: Topic, Result
  - Output: None (writes to JSON file)
  - Handler: save_to_history(topic, result)
- **Delete History API**
  - Description: Removes a specific history entry
  - Input: Entry ID
  - Output: Updated history
  - Handler: delete_history_entry(entry_id)

3 Data Design

The system uses a simple file-based data design for storing outputs and history.

- Study guides are stored as Markdown (.md) files
- Metadata is stored in a JSON file (history.json)
- Each entry includes ID, topic, result, timestamp, and preview
- Data is organized for easy retrieval and display in the frontend

3.1 Data Model

The system stores basic information for each generated study guide:

- ID – Unique identifier
- Topic – User input
- Result – Generated content
- Timestamp – Creation time
- Preview – Short summary

Data Structure (JSON Format)

JSON

```
{
  "id": "20260323_123456",
  "topic": "Machine Learning",
  "result": "Generated study guide content...",
  "timestamp": "March 23, 2026 12:34",
  "preview": "Short preview of content..."
}
```

3.2 Data Access Mechanism

The system employs a lightweight, file-based data access mechanism to manage generated content and metadata. Data operations are handled through dedicated Python functions that ensure controlled reading and writing of files.
Study guides are stored as Markdown (.md) files, while metadata such as topic, timestamp, and preview is maintained in a JSON file (history.json**)**. Functions like load_history(), save_to_history(), and delete_history_entry() are used to perform create, read, and delete operations.
The data access process is synchronous and optimized for simplicity, making it suitable for small to medium-scale applications while ensuring easy retrieval and management of stored content.

3.3 Data Retention Policies

The system follows a simple data retention approach to manage stored content efficiently.

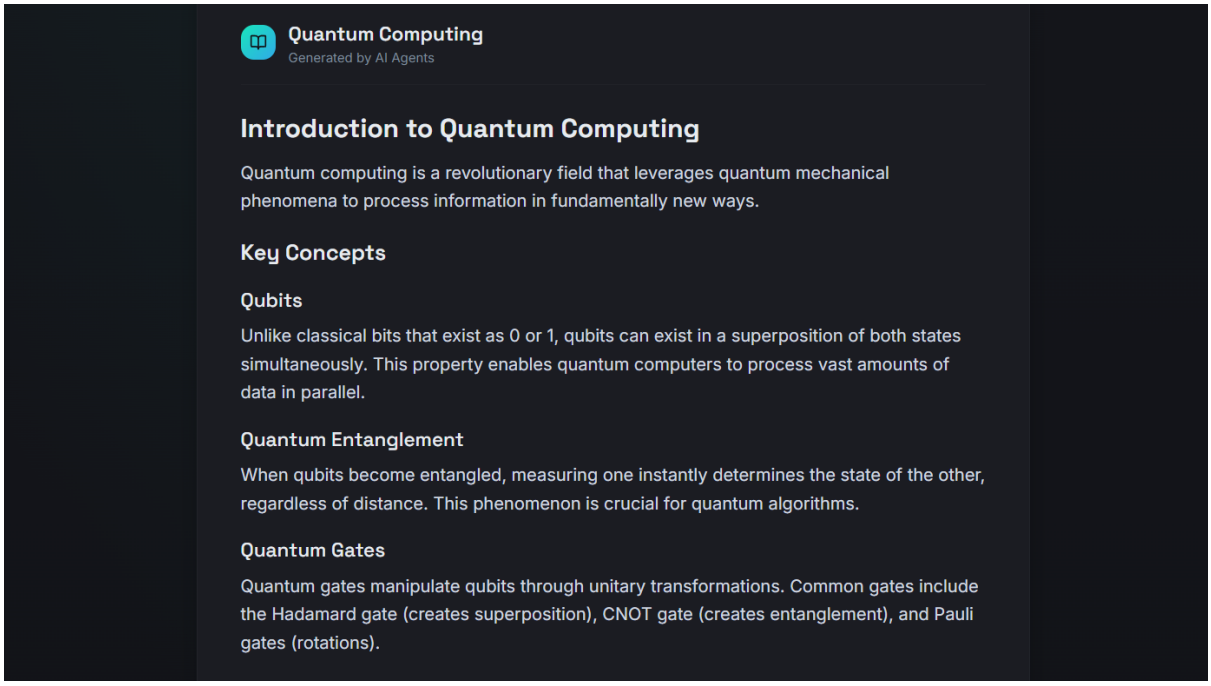
- Generated study guides are stored locally as markdown files for future reference
- A limited number of recent entries (e.g., last 30) are maintained in the history file
- Older entries are automatically removed to control storage size
- Users have the option to delete specific entries or clear all history
- No long-term archival or backup mechanism is currently implemented

3.4 Data Migration

The current system uses a simple file-based storage approach and does not require complex data migration mechanisms.

- Data is stored locally as markdown files and JSON records
- In case of system updates or scaling, data can be migrated to a database (SQL/NoSQL)
- Migration can be performed by converting existing JSON records into structured database entries
- Markdown files can be retained or moved to cloud/object storage
- The design allows future migration with minimal changes to the system

4 Interfaces



5. State and Session Management

The system uses a lightweight approach for managing state and user sessions through Streamlit's session state. Temporary data, such as the currently generated result and selected history, is stored within the session to maintain continuity during user interaction. Each session operates independently and does not persist once the session ends. For long-term storage, the system relies on local files, including markdown files and a JSON-based history. Advanced session management features such as user authentication or distributed session handling are not implemented in the current design.

6 Caching

The system uses minimal caching and processes each request independently. Generated outputs are stored as markdown files and in JSON history for basic reuse. No dedicated caching mechanism is implemented, but it can be added in the future to improve performance.

7. Non-Functional Requirements

7.1 Security Aspects

The system ensures basic security by using environment variables to store sensitive information such as API keys. No user authentication or authorization mechanisms are currently implemented. Data is stored locally, reducing exposure to external threats, but advanced security features can be added in future versions.

7.2 Performance Aspects

The system processes requests sequentially, which ensures consistent output but may increase response time for larger inputs. Performance is dependent on the LLM response time. The lightweight design ensures smooth operation for small to medium workloads, and performance can be improved in the future using caching or parallel execution.

8. References

1. CrewAI Documentation. Available at: https://docs.crewai.com
2. Groq API Documentation. Available at: https://console.groq.com/docs
3. Streamlit Documentation. Available at: https://docs.streamlit.io
4. Python Software Foundation. *Python Documentation*. Available at: https://docs.python.org/3
5. Meta AI. *LLaMA 3.1 Model Documentation*. Available at: https://ai.meta.com